

TP git

Préparation

Installation de git >= 2.23

Nous utiliserons les commandes **restore** et **switch** introduites dans la version 2.23 de **git**, ce sont des alternatives aux commandes historiques **checkout** et **reset** qui sont surchargées et causent trop de confusions.

1. Installez **git** et vérifiez que la commande **git --version** retourne une version supérieure à 2.23.

Installation de tig et gitk (facultatif)

De nombreuses interfaces graphiques permettent de visualiser l'historique et l'état du dépôt.

Le paquet **tig** fournit une interface **curses** permettant une telle visualisation dans votre terminal.

1. Installez le paquet **tig**

Le paquet **gitk** fournit une interface Tk permettant une telle visualisation dans une fenêtre graphique séparée.

2. Installez le paquet **gitk**

Configuration

Les commits contiennent le nom et l'adresse mail de la personne qui l'a créé (cette personne est appelée *committer*).

1. Pour que les commits vous identifient correctement comme étant l'auteur et/ou le committer, tapez les commandes suivantes :

```
$ git config --global user.name "<Prénom> <Nom>"
$ git config --global user.email "<adresse_mail>"
```

Utilisez votre adresse mail universitaire.

2. Si votre version de **git** est au moins 2.28 et si vous voulez que la branche par défaut des dépôts que vous allez créer s'appelle **main** plutôt que **master**, tapez la commande :

```
$ git config --global init.defaultBranch main
```

3. Observez le résultat dans le fichier `~/.gitconfig`.

Solo linéaire : création du dépôt et premier commit

1. Relisez les slides jusqu'au slide « Boucle simple (solo local linéaire) » (page 15).
2. Sur votre système local, dans votre répertoire d'administration système ou d'UE projet, créez un répertoire nommé **tp-git/** et initialisez-y un dépôt **git**.
3. Observez que le répertoire **tp-git/** contient un répertoire caché nommé **.git/**.
4. Dans ce répertoire **tp-git/**, créez un fichier nommé **plop** dont le contenu est la chaîne **hop**.
5. Ajoutez le fichier **plop** à la staging area (index).

6. Créez un commit dont le commentaire est « mon premier commit ».

Solo linéaire : deuxième commit et observation

1. Relisez les slides « Informations » (pages 16 et 17).
2. Modifiez le fichier `plop` en remplaçant la chaîne de caractères `hop` par `hap`.
3. Observez ce que retournent les commandes `git status` et `git diff`, et familiarisez-vous avec le format du texte retourné par ces commandes.
4. Ajoutez le fichier `plop` à la staging area (index).
5. Observez ce que retournent les commandes `git status` et `git diff`. Expliquez pourquoi la seconde commande ne retourne rien.
6. Créez un fichier `toto` dont le contenu contient 4 lignes de votre choix, chaque ligne doit être une phrase avec au moins trois mots.
7. Ajoutez le fichier `toto` à la staging area (index).
8. Modifiez la 3e ligne du fichier `toto`.
9. Prévoyez ce que va renvoyer la commande `git status` ainsi que chacune des trois commandes `git diff` du slide « Informations » et vérifiez vos hypothèses en les exécutant.
10. Notez que pour vous limiter aux changements du fichier `toto`, il est possible d'ajouter ce nom de fichier comme argument aux commandes `git diff`.
11. Ajoutez le fichier `toto` à la staging area (index).
12. Observez ce que retourne chaque commande du slide "Informations".
13. Commitez vos changements.
14. Contemplez l'historique de vos commits avec la commande `git log`.
15. Utilisez les interfaces graphiques pour observer les apports et retraits des différents commits.

Solo linéaire : oups

1. Relisez les slides « Annuler les changements » (pages 18 et 19).
2. Éditez le fichier `toto`, supprimez sa première ligne et sauvegardez-le.
3. C'est une boulette, récupérez le contenu de la version précédente du fichier `toto` grâce à `git`.
4. Éditez le fichier `toto`, modifiez sa 4e ligne et sauvegardez-le.
5. Ajoutez le fichier `toto` à la staging area.
6. C'est encore une boulette, nettoyez la staging area ainsi que votre répertoire de travail de sorte à ce que le fichier `toto` soit identique à celui du dernier commit.

Solo linéaire : bilan

1. Assurez-vous que vous comprenez ce que sont le « worktree », la « staging area » et le « repository » et que vous connaissez les commandes du slide « Solo local linéaire : bilan » (page 20).

Solo programmation

Cette partie n'est pas prioritaire (vous pouvez passer au prochain paragraphe et y revenir plus tard).

1. Lisez la page du slide intitulée « Ne versionner que le source, ignorer les sous-produits » (page 21).
2. Créez un répertoire nommé `programmation/` dans votre répertoire de travail.
3. Créez-y un fichier de type "Hello world" nommé `hello.c`, et commitez-le (ici, le verbe « commiter » est un raccourci pour dire « ajouter à la staging area puis commiter »).
4. Compilez-le et exécutez-le.
5. Que donne la commande

```
$ git status
```

Vous ne voulez pas commiter les fichiers compilés, seulement les fichiers source. Mais vous ne voulez pas non-plus que ces fichiers polluent toutes les commandes d'observation de `git`. Nous allons dire à `git` d'ignorer ces fichiers.

6. Créez un fichier `.gitignore` à la racine de votre répertoire de travail (worktree) de sorte à éviter de commiter par inadvertance le fichier compilé.
7. Que donne la commande

```
$ git status
```
8. Si les fichiers compilés n'apparaissent plus, commitez le fichier `.gitignore`.
9. Modifiez le fichier `hello.c` de sorte à ce qu'il écrive plutôt "Hello git".
10. Compilez-le et exécutez-le.
11. Commitez-le.

Solo branches

1. Créez une branche nommée `essai` pointant sur le commit courant.
2. Allez sur cette branche (*i.e.* faites de `essai` votre branche courante).
3. Créez un fichier `test.txt` à la racine de votre répertoire de travail, ajoutez-le à la staging area et commitez.
4. Créez un tag nommé `milieu-test` sur le commit courant.
5. Modifiez le fichier `test.txt`, ajoutez-le à la staging area et commitez.
6. Par curiosité, regardez le contenu du fichier `.git/HEAD` et des différents fichiers et sous-répertoires du répertoire `.git/refs/` et essayez (rapidement) de comprendre comment `git` gère ses références (tags, branches, branche courante).
7. Retournez sur la branche `main`.
8. Modifiez le fichier `plop`, ajoutez-le à la staging area et commitez.
9. Mergez la branche `essai` dans la branche courante `main`.
10. Utilisez la commande `git log` pour connaître les différents hash des différents commits (ne notez que les 4 premiers caractères, pourquoi est-ce largement suffisant ?).
11. Faites un dessin du DAG des commits en indiquant les différentes références (le tag `milieu-test`, la branche `essai`, la branche `main`, la tête `HEAD`).
12. Observez les dessins produits par `tig` et `gitk`, et assurez-vous que les commits pointés par les différentes références sont bien ce à quoi vous vous attendiez.

Solo merge manuel

Dans le paragraphe précédent, le merge a été résolu automatiquement car les modifications des deux branches `main` et `essai` ont concerné des fichiers différents. Ici, on va s'entraîner à résoudre un merge à la main.

1. Créez deux branches nommées `modif1` et `modif2` pointant sur le commit courant.
2. Faites de `modif1` votre branche courante.
3. Modifiez le premier mot de la deuxième ligne du fichier `toto`.
4. Commitez vos changements.
5. Faites de `modif2` votre branche courante.
6. Modifiez le dernier mot de la deuxième ligne du fichier `toto`.
7. Commitez vos changements.
8. Mergez la branche `modif1` dans la branche courante (`modif2`).

Un message d'erreur apparaît parce que `git` n'est pas capable de choisir quelle modification est la bonne sur la deuxième ligne du fichier `toto`. Il faut résoudre le merge à la main. On va supposer que les deux modifications sont bonnes, autrement dit, on veut une nouvelle version (*i.e.* un nouveau commit) qui

prene en compte les modifications faites sur le premier mot et sur le dernier mot de la deuxième ligne du fichier `toto`.

9. Pour voir ce qu'il vous reste à faire, utilisez la commande `git status`.
10. Éditez le fichier `toto`.

Les deux modifications apportées à la deuxième ligne sont encadrées par des `<<<<` et des `>>>>`. On observe 3 versions de la deuxième ligne. Modifiez ce qui se trouve dans cette zone de sorte à ce que la deuxième ligne du fichier `toto` prenne en compte les deux changements effectués sur la deuxième ligne.

11. Ajoutez le fichier `toto` à la staging area.
12. Pour voir ce qu'il vous reste à faire, utilisez la commande `git status`.
13. Commitez votre changement (vous pouvez utiliser le mot « Merge » comme message de commit).
14. Faites un `git status` pour vous assurer que le merge est fini.
15. Retournez sur la branche `main`.
16. Mergez `modif2` dans `main`.
17. Observez la situation avec `gitk` ou `tig` et constatez qu'aucun nouveau commit n'a été créé. Expliquez pourquoi.

Duo dessine moi un DAG

Cet exercice est à faire en compagnie d'un-e camarade (vous pouvez passer au prochain paragraphe et y revenir plus tard).

1. Demandez à votre camarade de dessiner sur une feuille un DAG un peu complexe avec des tags et des branches à plusieurs endroits.
2. Amusez-vous à réaliser ce DAG comme le DAG des commits de votre dépôt avec les commandes `add`, `commit`, `branch`, `switch`, `tag`, `merge`.

Si vous ne voulez pas perdre de temps à créer du contenu à commiter mais vous concentrer sur la gestion du DAG, vous pouvez au choix (ou successivement) :

- dessiner directement votre DAG dans le simulateur `learnitbranching`
- utiliser les alias suivants (le premier crée un fichier vide avec un nom aléatoire, l'ajoute dans la staging area et le commit avec un message aléatoire, le second dessine le DAG des commits en ASCII-art) :

```
$ alias Commit_un_truc='R=$RANDOM;touch f$R;git add f$R;git commit -m c$R'
$ alias DAG='git log --graph --oneline --all --decorate --color'
```

Ainsi, il suffit de taper la commande `Commit_un_truc` (utilisez l'autocomplétion !) pour créer un nouveau commit et la commande `DAG` pour observer le DAG obtenu.

3. Demandez à votre camarade de choisir une branche et un de ses ancêtres lointains.
4. Faites de l'ancêtre choisi le commit courant sans utiliser son hash, mais avec l'adressage relatif à la branche choisie (cf slide « Nommage relatif des ancêtres d'un commit » page 26).
5. Cherchez ensemble des commits qui possèdent des ancêtres joignables par plusieurs chemins orientés dans le DAG. Faites un `git diff` entre ces deux adresses du même commit pour vérifier qu'il n'y a pas de différence (et valider ainsi votre hypothèse).
6. Imaginez d'autres challenges de ce type de sorte à devenir à l'aise dans la navigation dans le DAG des commits. Amusez-vous. Si vous découvrez un exercice instructif, n'hésitez pas à le partager sur le salon `sysadmin_git` du `mattermost`.
7. Inversez les rôles.

Collectif : constituer une équipe et cloner un dépôt commun

1. Votre groupe de projet sera votre équipe pour la suite de ce TP. Si vous n'avez pas de groupe projet, il devient urgent d'en trouver ou d'en créer un !
2. L'enseignant-e de sysadmin a laissé sur le salon mattermost de votre équipe le nom de votre dépôt distant sur lequel travailler collectivement (si vous n'avez pas reçu de message à ce sujet, vérifiez que l'enseignant-e est invité sur votre salon mattermost de votre équipe ou invitez-le).
3. Voici quelques informations sur le système hébergeant le dépôt distant. Lorsque nous serons plus avancé-es dans le cours de sysadmin, toutes ces informations devraient vous être familières et compréhensibles. Pensez à revenir dessus.
 - le système distant est un conteneur accessible via le nom de domaine `yologit.netlib.re`.
 - elle possède sa propre adresse IPv6. Comme vous ne savez pas encore comment joindre un système IPv6-only par SSH depuis un réseau IPv4-only, ce conteneur hérite de l'adresse IPv4 du système hôte qui l'héberge, grâce à une redirection de port. Ainsi, le port de connexion SSH ne sera pas le port 22 (défaut), mais 2271.
 - les fingerprints des clefs publiques du serveur SSH de ce conteneur sont :
 - RSA : SHA256:uwcsZ4oiL19jikWYFA2S6u3DhF/NUSnnHRP1wJjeBzk
 - ECDsa : SHA256:bAQxBoag6n0vmmt657sMzP45ktbe/1TKnVshSNAJnyE
 - ED25519 : SHA256:0eoJohNR3VCQBP70xHORYdQ8720cXobNJNTr32Focba
 - l'utilisateur qui héberge les dépôts distants sur ce conteneur s'appelle `gituser`.
 - le shell de l'utilisateur `gituser` a été configuré pour n'accepter que des commandes `git`. En effet, le fichier `/etc/passwd` du système `yologit.netlib.re` contient la ligne :
`gituser:x:1000:1000:::/home/gituser:/usr/bin/git-shell`
 - si vous avez eu accès à un conteneur pendant le cours de sysadmin, votre clef publique SSH a été installée dans le fichier `/home/gituser/.ssh/authorized_keys` du système `yologit.netlib.re`. Sinon, il est possible de s'authentifier par mot de passe, celui-ci se trouve dans l'en-tête du salon mattermost `sysadmin_git`, il est n'est pas public.
4. Retournez dans votre répertoire d'administration système ou d'UE projet (quittez `tp-git/`) et clonez le dépôt distant qui vous a été attribué :

```
$ git clone ssh://gituser@yologit.netlib.re:2271/~/<nom_du_depot_distant>
```
5. Un message d'alerte s'affiche. Afin d'être sûr-e que la connexion n'a pas été interceptée par une entité malveillante, vérifiez que le fingerprint du serveur SSH du système distant correspond bien à l'un des fingerprints cités plus haut (point 3).
6. Une fois que vous avez vérifié le fingerprint, vous pouvez taper `yes`, et `git` clonera le dépôt via SSH.
7. Allez dans le répertoire de travail de votre nouveau dépôt. Tapez la commande

```
$ git remote -v
```

et observez que votre dépôt connaît un dépôt distant qu'il a nommé `origin`.
8. Par curiosité, observez ce qui est apparu dans le fichier `.git/config` et dans le répertoire `.git/refs/`.

Collectif : travailler en parallèle sur des fichiers disjoints

1. Dans le répertoire de travail de votre dépôt local fraîchement cloné se trouve un répertoire nommé `perso/`. Créez-y un fichier dont le nom est votre `<prénom>.<nom>` et ajoutez-y des informations (non compromettantes) sur vous (n'hésitez pas à baratiner, pensez que la confidentialité de ce dépôt est très faible).
2. Commitez ces changements et poussez-les sur le dépôt distant. Ne créez pas de nouvelle branche, restez sur la branche `main`.
3. Recommencez l'opération en rajoutant des détails dans le fichier `perso/<prénom>.<nom>` et repoussez votre nouveau commit.

4. Recommencez jusqu'à devoir merger les changements effectuées par un-e autre étudiant-e de votre équipe.
5. Continuez jusqu'à ce que tout le monde ait pu faire des **fetch** des **push** et des **merge**. Assurez-vous de vous être vu-e refuser au moins une fois un **push** car une autre personne a pushé sur la branche **main** entre votre dernier **fetch** et votre dernier **push**. Si nécessaire, laissez du temps à vos camarades avant de pusher vos nouveaux commits. Notez que les **merge** ne poseront pas de problème puisque les modifications sont faites sur des fichiers différents.
6. Observez l'historique et le DAG des commits.

Collectif : travailler en parallèle sur des parties distinctes d'un même fichier, avec plusieurs branches et un-e release manager

Le but de ce paragraphe est de jouer au jeu du *cadavre exquis* inventé par les surréalistes, mais avec **git**.

1. Désignez 5 joueu-ses et un-e maitre-sse du jeu (« release manager »). Le ou la release manager peut aussi être un-e joueu-se si vous n'êtes pas assez nombreu-ses. Dans tout le jeu, le ou la release manager est la seule personne qui peut modifier la branche **main**.
2. Dans le répertoire de travail de votre dépôt local partagé se trouve un répertoire nommé **cadavre_exquis/**. Dans ce répertoire se trouve un fichier nommé **template.txt**.
3. Le ou la release manager copie le template en un fichier nommé **jeu.1.txt**, crée un commit contenant ce fichier en restant sur la branche **main** et pousse cette branche sur le dépôt commun que tout-es les joueu-ses récupèrent.
4. Assurez-vous que tout le monde ait récupéré la branche **main** sur son dépôt local.
5. Répartissez les 5 lignes à remplir parmi les joueu-ses (1. substantif, 2. adjectif, 3. verbe, ...). Si vous n'êtes pas assez nombreu-ses dans votre équipe, attribuez deux lignes à une même personne (mais c'est moins drôle).
6. Une fois que vous êtes d'accord sur la ligne que chaque joueu-se doit remplir, créez chacun-e une branche ayant pour nom votre prénom (les branches doivent avoir des noms différents), et faites-en votre branche courante.
7. Chaque joueu-se modifie en secret le fichier **jeu.1.txt** en ajoutant au niveau du tiret qui correspond à sa ligne un ensemble de mots admissibles (par exemple un verbe pour la 3e ligne). Évitez les termes offensants ou discriminatoires.
8. Chaque joueu-se commite ses changements sur sa branche et pousse sa branche sur le dépôt commun.
9. Le ou la release manager récupère toutes les branches sur son dépôt local, les merge dans la branche **main** et pousse cette branche sur le dépôt commun. Comme les lignes modifiées sont différentes, il ne devrait pas y avoir de collision à gérer.
10. Tout le monde récupère la branche **main** du dépôt commun et découvre le résultat du cadavre exquis.
11. Observez qui a effectué les changements sur le fichier avec la commande :

```
$ git blame jeu.1.txt
```
12. Recommencez une partie (en remplaçant **jeu.1.txt** par **jeu.2.txt**, etc) de sorte à ce que tout le monde ait pu jouer le rôle de release manager.

Collectif : migrer vers gitlab

Gitlab est une forge logicielle installée à l'institut Galilée. Elle va vous permettre d'héberger vos dépôts distants (en: remotes) sans effort (le cours d'administration système devrait vous rendre capable d'héberger vous-même vos dépôts distants).

Son adresse est <https://git.ig-edu.univ-paris13.fr/>

1. Connectez-vous sur le gitlab de l'institut Galilée avec vos identifiants universitaires.
2. Créez un groupe gitlab au nom de votre équipe (cherchez « Nouveau groupe » (en: New group) dans la barre du haut).
3. Ajoutez les membres de votre équipe au groupe gitlab (cherchez le lien « Inviter vos collègues » (en: Invite your colleagues)).
4. Créez un dépôt partagé, dont le nom correspond au dépôt collectif utilisé dans les parties précédentes :
 - cliquez sur « Nouveau projet » (en: New repository, mal traduit) puis « Créer un projet vide » (en: Create blank project),
 - sélectionnez « Privé » (en: Private) pour « Niveau de visibilité » (en: Visibility Level) de sorte à ce que ce dépôt ne soit visible que par les membres de votre équipe,
 - décochez « Initialiser le dépôt avec un README » dans la section « Configuration du Projet » (ou « Initialize repository with a README » dans « Project Configuration ») de sorte à partir d'un dépôt vraiment vide.
5. Récupérez l'adresse web de ce dépôt : cliquez sur le bouton « Code » (en: Clone, mal traduit) en bleu et notez l'URL indiquée sous « Cloner avec HTTPS » (en: Clone with HTTPS) (elle est de la forme `https://git.ig-edu.univ-paris13.fr/<nom_groupe>/<nom_dépôt>.git`).
6. Ajoutez ce nouveau remote sur votre dépôt local et donnez-lui le nom gitlab :

```
$ git remote add gitlab https://git.ig-edu.univ-paris13.fr/<nom_groupe>/<nom_dépôt>.git
```
7. Vérifiez que votre dépôt local connaît les dépôts distants `origin` et `gitlab` :

```
$ git remote -v
```

Par curiosité, observez ce qui est apparu dans le fichier `.git/config` et dans le répertoire `.git/refs/`.
8. Désormais, lorsque vous pushez ou fetchez vos changements, utilisez le remote `gitlab` plutôt que `origin`, par exemple :

```
$ git fetch gitlab main
$ git push gitlab main
```

Collectif : travailler en parallèle sur des parties communes d'un même fichier en étant sur une même branche

L'intégralité de cet exercice se fait sur la branche `main`.

1. Dans le répertoire de travail de votre dépôt local, créez un fichier nommé `baston`, commencez à y écrire un texte, commitez-le et poussez vos changements sur le dépôt distant.
2. Recommencez l'opération en rajoutant des détails à votre texte dans le fichier `baston` et repoussez votre nouveau commit.
3. Modifiez des lignes existantes pour ajouter des précisions.
4. Recommencez jusqu'à devoir intégrer les changements effectués par un-e autre étudiant-e de votre équipe.
5. Lorsque vous mergez, assurez-vous de maintenir la cohérence globale du texte (le sens peut évoluer mais il doit toujours avoir du sens et être grammaticalement correct) sans discuter entre vous.
6. Si un `merge` doit être résolu manuellement, décidez seul-e de la version à garder pour que le texte reste cohérent.
7. Continuez jusqu'à ce que tout le monde ait pu faire des `fetch`, des `push` et des `merge` avec résolution manuelle en rapport avec le fichier `baston`.
8. Discutez des difficultés que vous avez rencontrées.

Collectif : merge request

Un workflow classique adapté à recevoir les contributions d'inconnu-es n'ayant pas accès au dépôt central est le modèle « merge request » (ou « pull request ») :

- Seul-e le, la ou les release managers peuvent écrire sur le dépôt central, dont la branche **main** ne contient que du code approuvé.
 - Chaque participant-e possède son propre dépôt distant et pushe uniquement sur ce dépôt.
 - Chaque participant-e travaille sur une branche dont le nom est la fonctionnalité concernée (par exemple **deplacement_robot** ou **affichage_robot**) et la pushe régulièrement sur son dépôt distant personnel.
 - Lorsque cette personne est satisfaite du résultat, elle contacte le, la ou les release managers pour que cette branche soit mergée dans la branche **main** du dépôt central.
 - Si le code est correct (processus de validation à définir collectivement, en particulier s'assurer que tous les tests passent), le, la ou les release managers mergent la branche candidate dans la branche **main** et pushent cette branche sur le dépôt central.
 - Ensuite, tout le monde récupère régulièrement la branche **main** sur son dépôt personnel local. Si une personne travaille sur une branche alors que la branche **main** a été modifiée, elle peut merger la branche **main** sur sa branche locale et résoudre les problèmes avant de soumettre sa branche pour faciliter le travail du ou des release managers.
1. Rejouez au cadavre exquis avec ce workflow, en utilisant comme branches le nom de la partie et de la ligne à modifier plutôt que le nom du ou de la joueuse, par exemple : **partie4.substantif1**, **partie4.adjectif1**, **partie4.verbe**, **partie4.substantif2**, **partie4.adjectif2** (il vous faudra créer votre propre remote **gitlab_perso**).

Collectif : choisissez votre workflow

Si vous êtes arrivé-e à ce paragraphe dans les 3h, félicitations !

1. Discutez avec votre équipe, et répartissez-vous des tâches avec des fichiers à modifier, des noms de branches, et éventuellement des rôles pour savoir qui merge quoi (ou pas).
2. Essayez de les réaliser et amusez-vous.
3. Partagez vos expériences en identifiant le pour et le contre de votre workflow sur le salon **mattermost sysadmin_git**.
4. Gardez en tête que git est un outil flexible qui n'impose pas d'organisation collective particulière, ni même de dépôt central, ni même de release manager, et que le meilleur workflow est celui que vous déciderez ensemble.